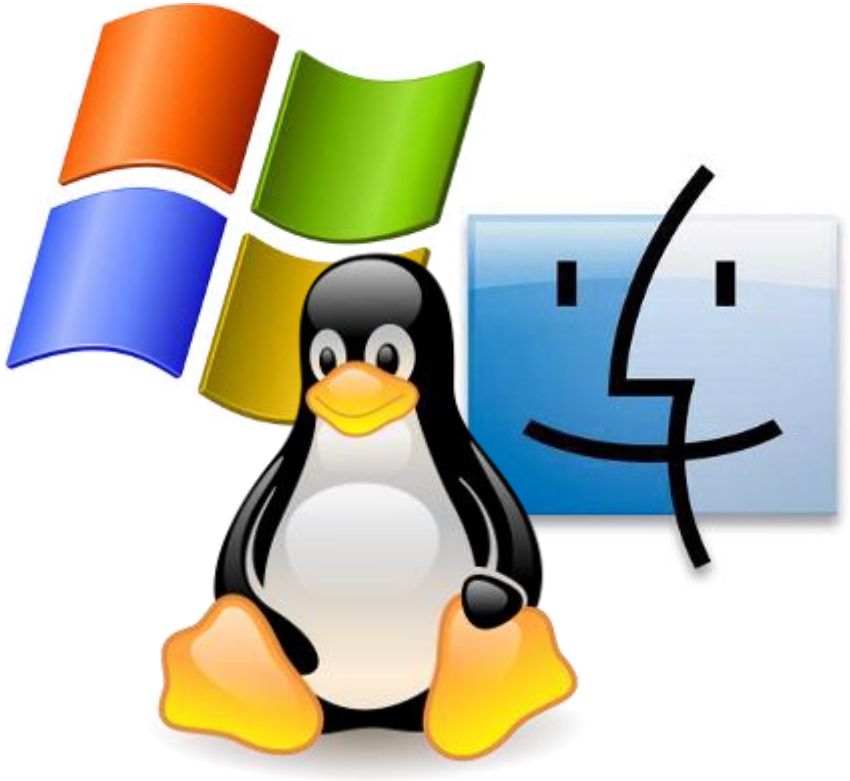
4 – Dateiverwaltung



Tobias Werner M. Sc.
Uni Bayreuth – Angewandte Informatik 3
Leibniz-FH Hannover

tobias.werner.doz@leibniz-fh.de

(in Teilen nach der Vorlesung
"Betriebssysteme" von Prof. Dominik Henrich
an der Universität Bayreuth)

[Logos von Windows 7, Linux und Mac OSX]

Einleitung

Heranführung

- Wie viele Dateien hat ein aktueller Computer?
- Wie erkläre ich einem Kollegen, wo auf einem Server eine Datei liegt?
- Eine Datei hat 64 Byte an Inhalt. Wie viel Platz braucht die Datei auf einem Datenträger?
- Was versteht man allgemein unter dem Begriff Fragmentierung?

Kompetenzen

- grundsätzliche Begriffe zum Umgang mit Dateien und Verzeichnissen erlernen
- typische Datenstrukturen der Dateiverwaltung in aktuellen OS kennen
- Vor- und Nachteile einzelner Verwaltungsansätze verstehen

Vortragsübersicht

4 – Dateiverwaltung

- Dateien
- Verzeichnisse
- Allokationsverfahren
- Praxisbeispiele

Dateien (1)

Motivation

beim Ablegen von Daten im Hauptspeicher stößt man schnell auf Probleme:

- Hauptspeicher ist begrenzt
  → Daten passen nicht komplett in den Hauptspeicher
- Hauptspeicher ist flüchtig
  → Daten gehen bei Prozessende oder System-Aus verloren
- Hauptspeicher ist pro Prozess virtualisiert
  → Daten sind zwischen Prozessen nur über Umwege teil- oder tauschbar
- Hauptspeicher ist prozesszentriert organisiert
  → Daten sind für (IT-ferne) Endnutzer schwer verständlich strukturiert

Lösungsansatz:

verwalte Daten in größerem, nicht flüchtigem und zwischen Prozessen geteiltem Speicher, der für (IT-ferne) Endnutzer verständlich organisiert ist

→ **Dateien** auf einem **Dateisystem**

Dateien (2)

Kernbegriffe

***Datei* (engl. *file*):**

Dateneinheit zur *persistenten* (dauerhaften) Speicherung von Daten auf Datenträgern

(z. B. auf Magnetband, Festplatte, CD, DVD, Speicherstick)

***Dateisystem* (engl. *file system*, *FS*):**

Strukturierung der Dateien und zugehöriger Verwaltungsinformationen auf Datenträgern

(z. B. FAT, NFTS, HFS, ZFS, ext2, ...)

***Dateiname* (engl. *file name*):**

Vergabe eines Namens für jede Datei in einem Dateisystem (z. B. durch den Benutzer)

→ Wiederauffinden der Datei über den Namen möglich

→ Freigabe des (alten) Namens beim Löschen oder Umbenennen der Datei

→ Beispiele: `"aufsatz.txt"`, `"octree_paper.pdf"`, `"2024-05-01 Geburtstagskuchen.jpg"`

Dateien (3)

Implementierungsdetails

wechselnde Regeln für Dateinamen:

- Welche Zeichentabelle wird verwendet? (z. B. ASCII, UTF-8, UCS2)
- Welche Namen oder Zeichen sind (un-)zulässig? (z. B. `"."`, `"-"`, `" "`, `"/"`, `"\"`)
- Wie lange darf ein Dateiname maximal sein? (z. B. 256 Zeichen in NTFS)
- Wird nach Groß-/Kleinschreibung unterschieden?

***Dateiendung* (engl. *file extension*):**

- Dateiname hat oft die Struktur `"name" + "." + "endung"`
- Endung kennzeichnet oft das Format des Inhalts der Datei
  (z. B. Text, JPEG-Bild, MP4-Video)
- Endung erlaubt Verknüpfung mit OS-Reaktion
  (z. B. Texteditor öffnen, Video abspielen)
- Endung ist in aktuellen FS nur Konvention
  → keine oder mehrere Endungen erlaubt
  → beliebig viele Buchstaben in Endung

Datei	Bedeutung der Endung
`Name.bak`	Backup-Datei
`Name.c`	C-Quelltextdatei
`Name.gif`	Bilddatei nach dem GIF-Standard
`Name.hlp`	Hilfedatei
`Name.html`	Hypertext-Markup-Language-Seite
`Name.jpg`	Bilddatei nach dem JPEG-Standard
`Name.mp3`	Musik nach MPEG Layer 3 kodiert
`Name.lnk`	Link-Datei (Endung i. d. R. versteckt)

Dateien (4)

Dateiarten

***gewöhnliche Dateien* (engl. *regular files*):**
dienen zur Speicherung von Benutzerdaten

schwammige Aufteilung regulärer Dateien in:

- *Textdateien*:
  → bestehen auschließlich aus Zeichen (z. B. enkodierten Buchstaben)
  → Zeichenenkodierung nach Zeichentabelle (z. B. ASCII)
  → Konvention für Zeilenenden hängt von OS ab (z. B. `"\n"`, `"\r\n"`, `"\r"`)
  → Inhalt meist mit einfacher Software menschenlesbar

- *Binärdateien*:
  → enthalten potentiell beliebige Folge beliebiger Bytes
  → Vorgabe der Semantik durch ein Dateiformat (was steht wo und wie?)
  → speichereffizienter als Textdateien
  → Inhalt nicht zwingend menschenlesbar

Dateien (5)

***Verzeichnisse* (engl. *directories*), manchmal auch O*rdner* (engl. *folders*):**
binäre Systemdateien zur Verwaltung des Dateisystems und anderer Dateien
→ Verzeichnisse sind vollständig unsichtbare Dateien im Überverzeichnis

***Gerätedateien* (engl. *device file*),**
manchmal auch *Spezialdateien* (engl. *special file*):
virtuelle Datei, die hineingeschriebene oder herausgelesene Daten speziell behandelt,
z. B.:

- *zeichenorientierte Spezialdateien* (engl. *character devices*):
  → übertragen Daten byteweise von einer Quelle zu einer anderen
  (z. B. zur Übertragung von Programmausgaben an ein Terminal)
- *blockorientierte Spezialdateien* (engl. *block devices*):
  → übertragen Daten, die in größeren Blöcken vorliegen müssen
  (z. B. zur Übertragung von Daten an externe Hardware)
- *Pseudo-Geräte* (engl. *pseudo devices*):
  → verhalten sich wie Datei mit algorithmisch generiertem Inhalt
  (z. B. zum Auslesen von beliebig vielen Zufallszahlen)

Dateien (6)

Aus [Tanenbaum15] Chapter 4.3.4 “Shared Files””

***Hard Link*:**

Verzeichniseintrag, welcher einen Dateinamen mit einer Datei assoziiert

→ erlaubt Ablage einer Datei in mehreren Verzeichnissen

→ Verwaltung z. B. über *Referenzzähler* oder die selben Metadaten

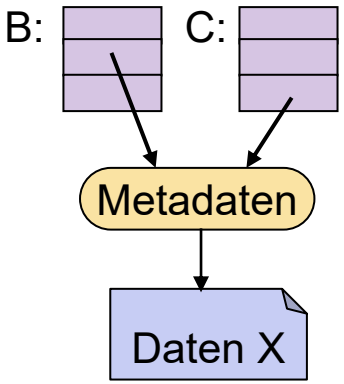
***Sym-Link* (*symbolic link*, manchmal auch *soft link*):**

Datei, die vom OS automatisch zu einer anderen Datei aufgelöst wird

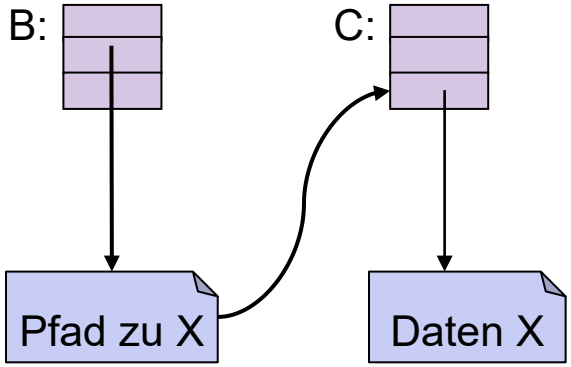
→ Speicherung der Zieldatei in einer extra Datei (*slow sym-link*) oder ...

→ ... in den Metadaten (*fast sym-link*)

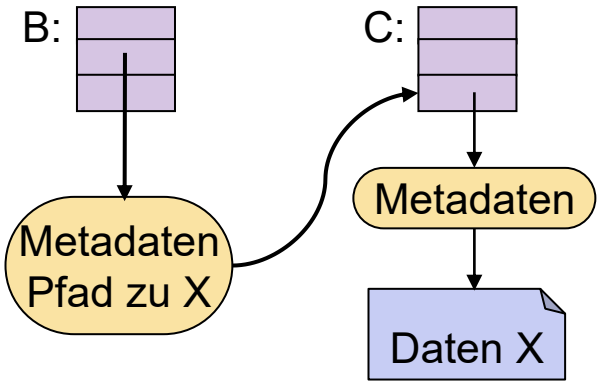
2 Hard-Links:



Slow Sym-Link:



Fast Sym-Link:



Dateien (7)

Dateiattribute

sind zusätzliche, **im Dateisystem** hinterlegte Metadaten zu Dateien

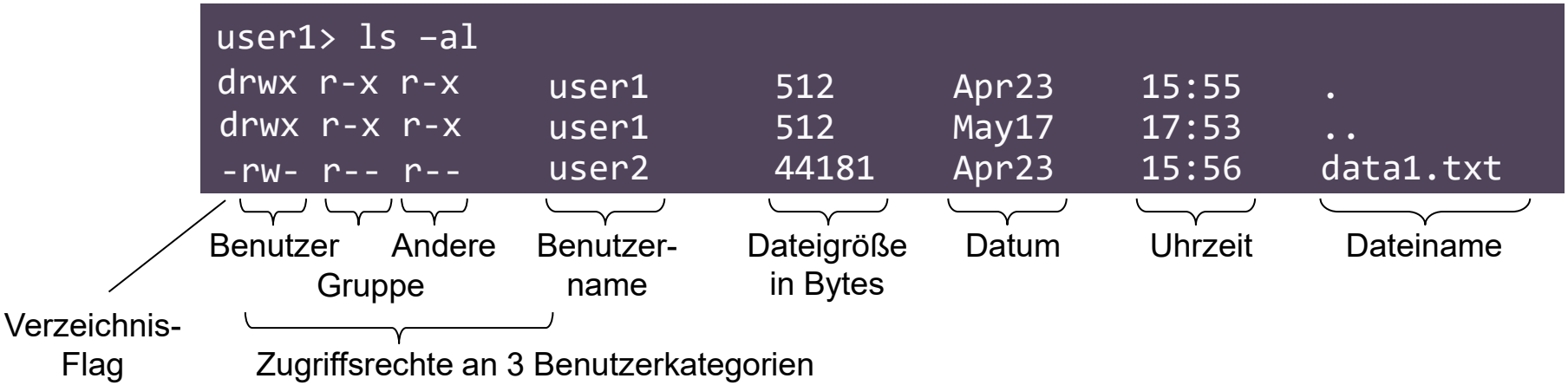
Beispiel Zugriffsrechte unter Linux:

Linux-Dateisysteme vergeben pro Datei an 3 Benutzerkategorien

jeweils 3 verschiedene *Zugriffsrechte*

→ Benutzerkategorien *u* (user), *g* (group), *o* (other)

→ Zugriffsrechte *r* (read), *w* (write), x (execute)



ähnlich: Erstelldatum, Änderungsdatum, Systemdatei-Flag, Versteckt-Flag, ...

Beispiel aus [Tanenbaum15]

Dateien (8)

Weitere Beispiele für Dateiattribute

Attribut	Bedeutung
Schutzinfos	wer darf wie auf die Datei zugreifen
Passwort	Passwort, das für den Zugriff benötigt wird
Erzeuger	Identifikation des Erzeugers der Datei
Eigentümer	aktueller Eigentümer
Read-Only-Flag	0 für Lesen/Schreiben, 1 für nur Lesen
Hidden-Flag	0 für normal, 1 für in Auflistungen nicht auftretend
System-Flag	0 für normale Datei, 1 für *Systemdatei* (d. h. OS-interne Datei)
Archive-Flag	0 für Sicherung erstellt, 1 für Sicherung muss gemacht werden
ASCII/Binary-Flag	0 für ASCII-Datei, 1 für Binärdatei
Random-Access-Flag	0 für nur sequentiellen Zugriff, 1 für wahlfreien Zugriff
Temporary-Flag	0 für normal, 1 für das Löschen bei Prozessende
Lock-Flag	0 für nicht gesperrt, nicht 0 für gesperrt
Erzeugung	Datum und Uhrzeit der Dateierzeugung
letzter Zugriff	Datum und Uhrzeit des letzten Zugriffs
letzte Änderung	Datum und Uhrzeit der letzten Änderung
aktuelle Größe	Anzahl der Bytes der Datei
Maximalgröße	Größe, bis zu der die Datei maximal wachsen darf

Dateien (9)

Dateizugriff

***sequentieller Dateizugriff* (*sequential access*):**

Inhalte von Dateien können nur der Reihe nach von Anfang an geschrieben oder gelesen werden

→ Beispiel: Magnetband zur Datensicherung

→ kein wahlfreies Herumspringen innerhalb der Datei möglich

→ Datenträger muss sonst kostenintensiv zurückgespult werden



[wikipedia]

***wahlfreier Dateizugriff* (*random access*):**

Inhalte von Dateien können (eventuell auch nur blockweise) in beliebiger Reihenfolge gelesen werden

→ Beispiel: Festplatten, USB-Sticks

→ Umsetzung über Zeiger für aktuelle Position

→ Funktionen `read()`, `write()` verschieben Positionszeiger entsprechend

→ Funktion `seek()` verschiebt Positionszeiger, ohne zu schreiben oder zu lesen

→ wahlfreies Herumspringen innerhalb der Datei mit mäßigem Aufwand möglich



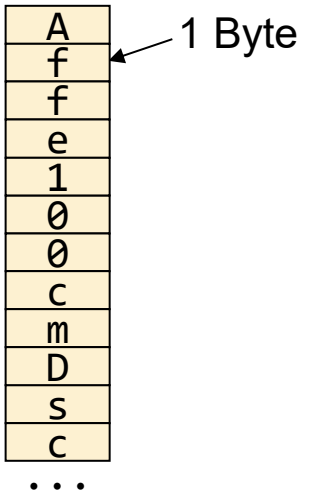
[wikipedia]

Dateien (10)

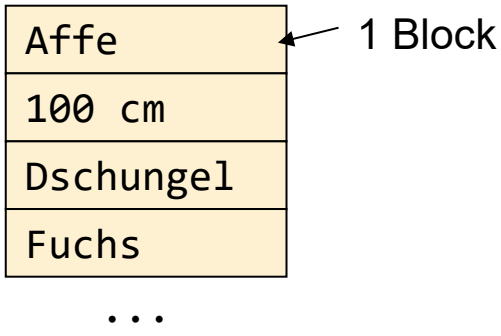
Dateistrukturen

Welche Struktur nutzt der Lese- und Schreibzugriff auf eine Datei?
(z. B. über eine API, über das OS, über die Hardware)

***byte-sequentielle Dateistruktur* (engl. *byte sequential files*):**
Zugriff auf Datei als lückenlose Folge von Bytes
→ in der Regel Zugriff auf beliebige Bytes erlaubt



***block-sequentielle Dateistruktur* (engl. *block sequential files*):**
Zugriff auf Datei als Folge disjunkter, meist gleich großer Blöcke
→ Zugriff (z. B. Schreiben) nur blockweise möglich



***hierarchische Dateistruktur* (engl. *hierarchical files*):**
Zugriff auf Datei über Baumstruktur aus Eltern- und Kindblöcken
→ effizientes Suchen und Austauschen von Teildaten möglich

aktuell: byte-sequentielle Systemaufrufe, block-sequentielle Hardware, APIs in allen drei Varianten je nach Aufbau der Datei

Aus [Tanenbaum15]

Dateien (11)

Dateioperationen

typische Dateioperationen als Systemaufrufe:

`create()`	→ erzeuge neue Datei mit gegebenem Namen und Pfad
`delete()`	→ lösche Datei und gebe Speicherplatz auf Datenträger frei
`rename()`	→ ändere Namen und Pfad der Datei
`open()`	→ öffne per Pfad gegebene Datei nach Wahl mit Lese- bzw. Schreibzugriff
`close()`	→ schließe offene Datei und gebe OS-interne Resourcen frei
`read()`	→ lese Bytes aus Datei in Hauptspeicher und verschiebe Positionszeiger
`write()`	→ schreibe Bytes aus Hauptspeicher in Datei und verschiebe Positionszeiger
`seek()`	→ verschiebe Positionszeiger
`getAttributes()`	→ frage Dateiattribute ab
`setAttributes()`	→ verändere Dateiattribute

Vortragsübersicht

4 – Dateiverwaltung

- Dateien
- Verzeichnisse
- Allokationsverfahren
- Praxisbeispiele

Verzeichnisse (1)

Ordnung für Dateien

Problem:

rechnerweit eindeutige Dateinamen führen schnell zu Konflikten

(z. B. bei häufigen Namen und über mehrere Programme oder Nutzer)

Lösung *Verzeichnis* (engl. *directory*),

auch *Ordner* (engl. *folder*):

dient der hierarchischen Gruppierung und Verwaltung von Dateien

→ ist in den meisten Systemen selbst als Datei realisiert

→ enthält typischerweise einen Eintrag pro Datei

→ Dateiname muss nur innerhalb des Verzeichnisses eindeutig sein

Öffnen von Dateien:

- OS durchsucht die Verzeichnisdatei, bis es den Namen der Datei findet, extrahiert die Attribute und die Plattenadresse und legt sie in einer Tabelle im Hauptspeicher ab (belegt als *Datei-Handle* interne OS-Resourcen)
- weitere Verwendung nutzt Informationen im Hauptspeicher

Verzeichnisse (2)

Verzeichnisstrukturen

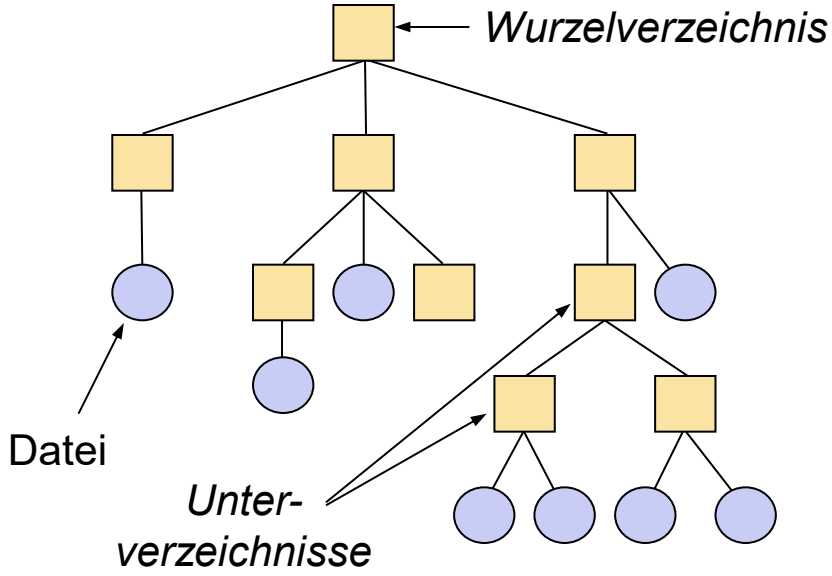
typische Implementierung:

allgemeiner Verzeichnis-Baum

→ ein *Elternverzeichnis* enthält *Unterverzeichnisse*

→ ein *Wurzelverzeichnis* hat kein Elternverzeichnis

→ ein Verzeichnis darf sich nicht selbst enthalten
(auch nicht über mehrere Unterverzeichnisse)



Funktionen:

- Nutzer-Rechte pro Datei bzw. pro Verzeichnis
- mehrere, gleichnamige Dateien pro Benutzer
- inhaltliche Sortierung der Dateien möglich

seltenere, ältere Varianten:

- ein Verzeichnis für den ganzen Rechner
- ein Verzeichnis pro Benutzer
- ein Verzeichnisbaum pro Benutzer

Verzeichnisse (3)

Pfadangaben

***Pfadname* (engl. *path*):**
spezifiziert auf hierarchischen Verzeichnisstrukturen den Ablageort einer Datei
als Folge von Verzeichnisnamen
→ *Trennzeichen* (engl. *separator*) zwischen Verzeichnissen kann variieren
(z. B. `/` unter Linux, `\` unter Windows, `>` unter MULTICS)

***absoluter Pfadname* (engl. *absolute path*):**
Angabe eines Ablageorts ausgehend vom Wurzelverzeichnis

Beispiele:

- Windows: Laufwerke (z. B. `C:`, `D:`) sind Wurzelverzeichnisse
  → ein absoluter Pfad wäre `C:\Windows\notepad.exe`

- Linux: Wurzelverzeichnis hat leeren Namen
  → ein absoluter Pfad wäre `/bin/cd`

Verzeichnisse (4)

***relativer Pfadname* (engl. *relative path*):**
Angabe eines Ablageorts ausgehend von einem beliebigen, anderen Verzeichnis

Arbeitsverzeichnis ist pro Prozess (z. B. per Systemaufruf) wählbares Verzeichnis, bezüglich dem relative Pfade standardmäßig aufgelöst werden

Pseudo-Verzeichnis erlaubt verzeichnisabhängige Sprünge:
→ Pseudo-Verzeichnis `".."` navigiert in übergeordnetes Verzeichnis
→ Pseudo-Verzeichnis `"."` navigiert in aktuelles (Arbeits-)Verzeichnis

Beispiele:
- Auflösung von relativem Windows-Pfad `images\cat.jpg` ergibt...
  ... bezüglich Arbeitsverzeichnis `C:\Programme\Photoshop` einen Zugriff
  auf die Datei `C:\Programme\Photoshop\images\cat.jpg`
  ... bezüglich frei gewähltem Verzeichnis `C:\Users\Werner\Desktop` einen Zugriff
  auf die Datei `C:\Users\Werner\Desktop\images\cat.jpg`
- relativer Windows-Pfad `dog\.\..\Steam\Steam.exe` löst bei vorhandenem Verzeichnis `dog` und vorigem Arbeitsverzeichnis immer auf zu Datei `C:\Programme\Steam\Steam.exe`

Verzeichnisse (5)

***kanonischer Pfadname* (engl. *canonical path*):**
genormte Angabe eines absoluten oder relativen Pfads ohne Pseudo-Verzeichnisse

Beispiele:

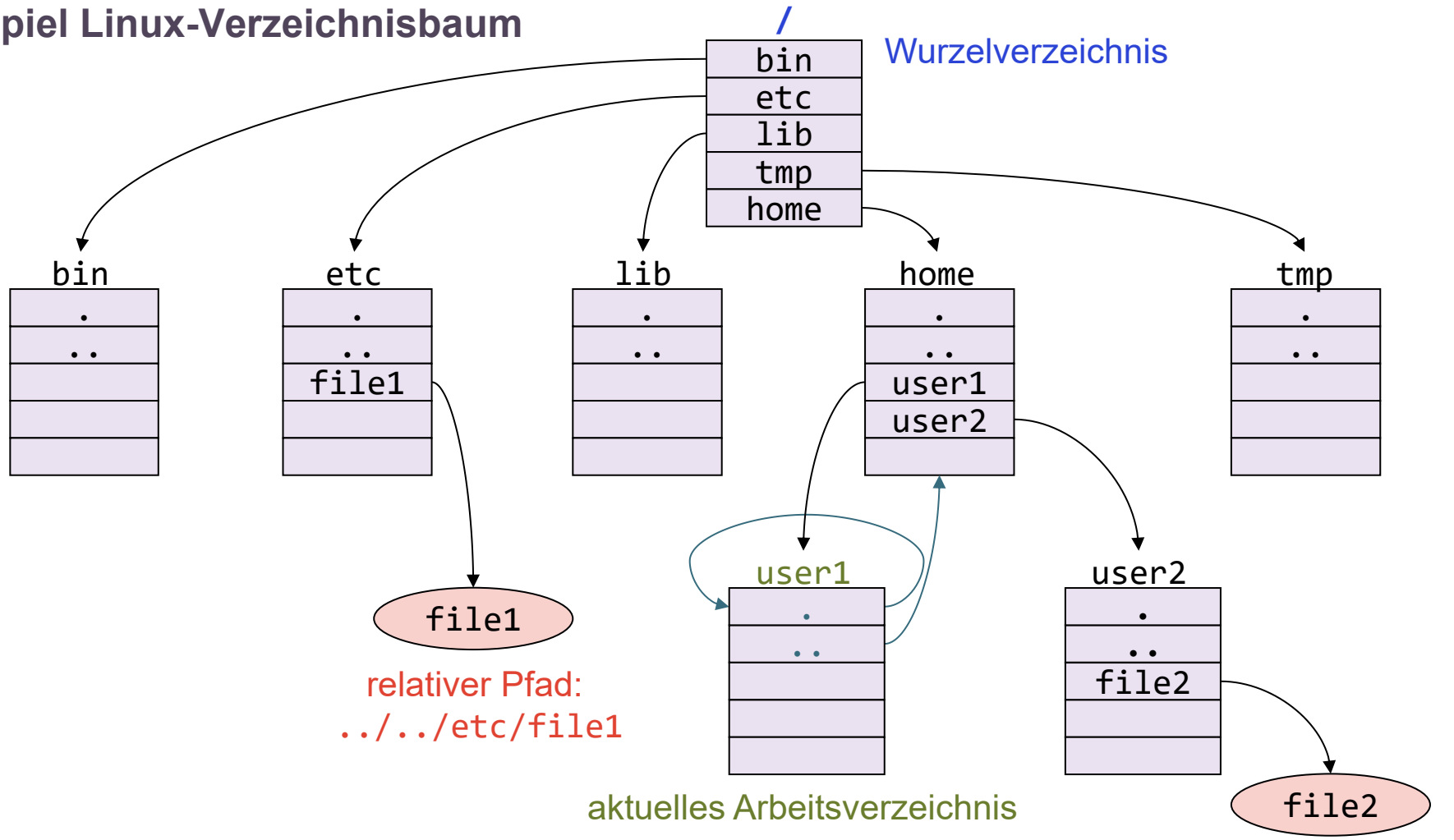
- Windows-Pfad `C:\Windows\notepad.exe` ist absolut und kanonisch
- Linux-Pfad `Documents/Texts/../Images/image.jpg` ist relativ und nicht-kanonisch

Anwendungsbeispiel zu kanonischen Pfaden:
zwei nicht-kanonische Pfade können die selbe Datei angeben
→ ein Vergleich der Verzeichnis-Strings genügt nicht, um das herauszufinden
→ wandle die Pfade vor dem Vergleich in kanonische Pfade um

Beispiel:
Windows-Pfade `C:\Programme\Photoshop\Libraries\..\Photoshop.exe`
und `..\Urlaubsbilder\.\Examples\..\Photoshop.exe` verweisen für das
Arbeitsverzeichnis `C:\Programme\Photoshop` auf den kanonischen Pfad
`C:\Programme\Photoshop\Photoshop.exe`

Verzeichnisse (6)

Beispiel Linux-Verzeichnisbaum



Aus [Tanenbaum15]

Verzeichnisse (7)

Verzeichnisoperationen

typische Verzeichnisoperationen als Systemaufrufe:

`create_dir()`	→ erzeuge neues, leeres Verzeichnis mit gegebenem Namen und Pfad
`delete_dir()`	→ lösche bestehendes Verzeichnis
`open_dir()`	→ öffne Verzeichnis, um darin enthaltene Dateien durchzugehen
`close_dir()`	→ schließe Verzeichnis und gib OS-interne Resourcen frei
`read_dir()`	→ frage die nächste Datei in geöffnetem Verzeichnis ab (liefert z. B. Dateinamen und Dateiattribute zurück)

Vortragsübersicht

4 – Dateiverwaltung

- Dateien
- Verzeichnisse
- Allokationsverfahren
- Praxisbeispiele

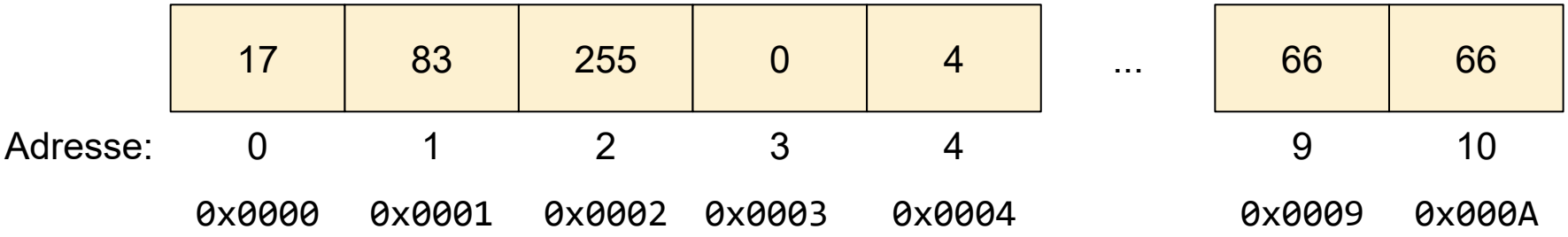
Allokationsverfahren (1)

Datenträgermodell für Betriebssysteme

- Datenträger besteht aus einer Folge von Speicherzellen
- Adressen und Werte wie beim Hauptspeicher (→ VL-Einheit 3)
- *adressierbarer Datenträgerspeicher*
  → hängt vom Dateisystem (und i. d. R. nicht von der CPU-Architektur) ab
  → typischerweise 2^{32} bis 2^{64} Byte
- *physisch verfügbarer Datenträgerspeicher*
  → hängt vom konkreten Datenträger ab (z. B. 1 TB auf einer Samsung 850 Pro-SSD)

Aufgabenstellungen:

- Wo legt das OS eine Datei in den Bytes des Datenträgers ab?
- Welchen Zugriff bietet das OS bzw. die Hardware auf die gespeicherten Daten an?



Allokationsverfahren (2)

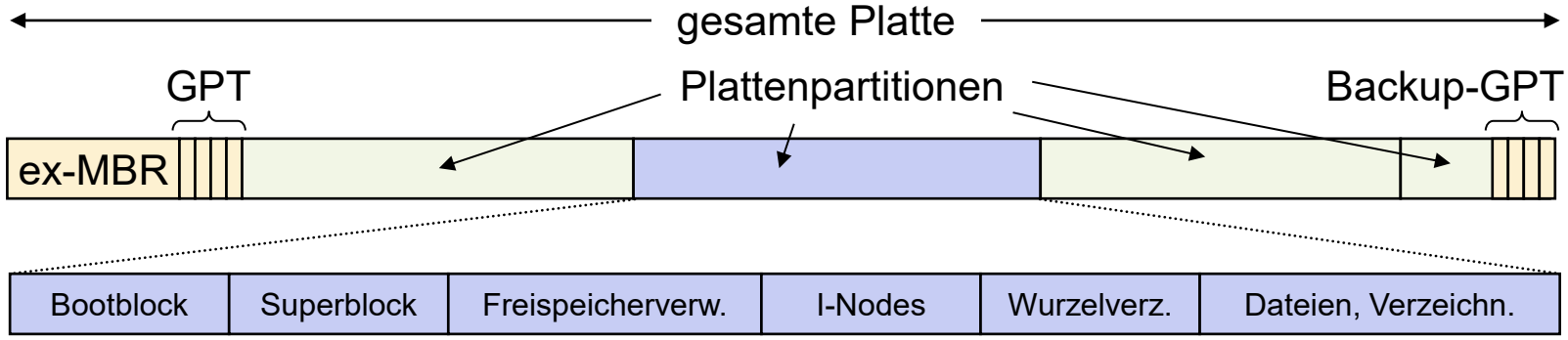
Datenträger und Partitionen

innere Struktur eines Datenträgers:

- mehrere unabhängige *Partitionen* (z. B. je eigenes Dateisystem, eigene Größe)
- *GUID-Partitionstabelle* (*GPT*) gibt Start- und Endadresse der Partitionen an
- *Master Boot Record* (*MBR*) ist veraltet und nur noch aus Sicherheitsgründen da

Beispielstruktur einer Partition:

- *Bootblock* enthält Einstiegsprogramm für OS
- *Superblock* speichert Parameter des Dateisystems
- *freie Blöcke* (z. B. je 512 kB) können Daten aufnehmen
- *I-Nodes* enthalten Metainfos zu jeder Datei



Allokationsverfahren (3)

Booten des Rechners

Idee: Rechner *bootet* (startet) beim Anschalten automatisch *Firmware* ("feste Software")

→ Firmware bootet als "virtuelle Maschine" über mehrere Schritte das echte OS

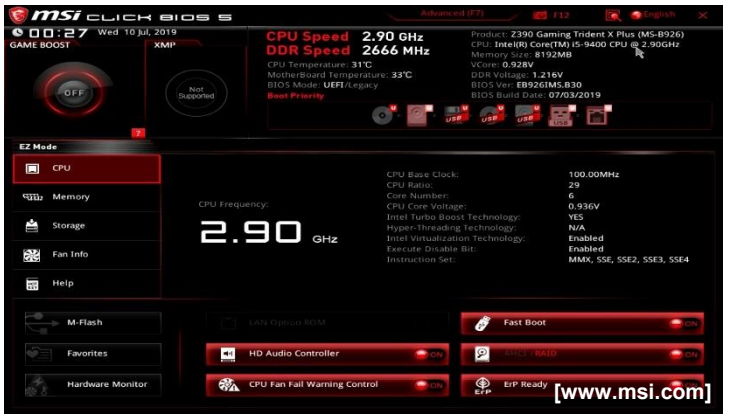
zwei typische Varianten für die Firmware bei Desktop-PCs:

- *Binary Input/Output System* (*BIOS*, seit ca. 2010 veraltet)
- *Unified Extensible Firmware Interface* (*UEFI*, ab ca. 2000 verfügbar)
  - → unterstützt Datenträger mit > 2 TB Speicher und GPT
  - → unterstützt Grafikdarstellung beim Bootprozess
  - → unterstützt Secure Boot

BIOS



UEFI



Allokationsverfahren (4)

Konzeptueller Startvorgang bei UEFI

(BIOS ähnlich)

1. UEFI prüft verfügbare Hardware (*power-on self-tests*, *POST*)
2. UEFI bestimmt vordefiniert oder über Benutzerwahl zu bootendes OS (bei GPT)
   oder zu bootenden Datenträger (bei MBR)

3. UEFI startet einen OS-spezifischen *Bootloader* (OS-Startprogramm):
   → Booten vom GPT-Datenträger:
   alle Bootloader für OSe liegen in separater EFI-Partition vor,
   UEFI startet daraus den gewählten OS-spezifischen Bootloader
   → Booten vom MBR-Datenträger:
   UEFI startet einen allgemeinen Bootloader im MBR des Datenträgers,
   der startet den OS-spezifischen Bootloader der aktiv markierten MBR-Partition

5. OS-spezifischer Bootloader startet OS
6. OS sucht für jedes Gerät den Geräte-Treiber und lädt den Treiber in den OS-Kern
7. OS erzeugt Verwaltungstabellen und Hintergrundprozesse
8. OS startet GUI mit Login

Allokationsverfahren (5)

Kontinuierliche Allokation

Speicherung jeder Datei als eine zusammenhängende Menge von Blöcken

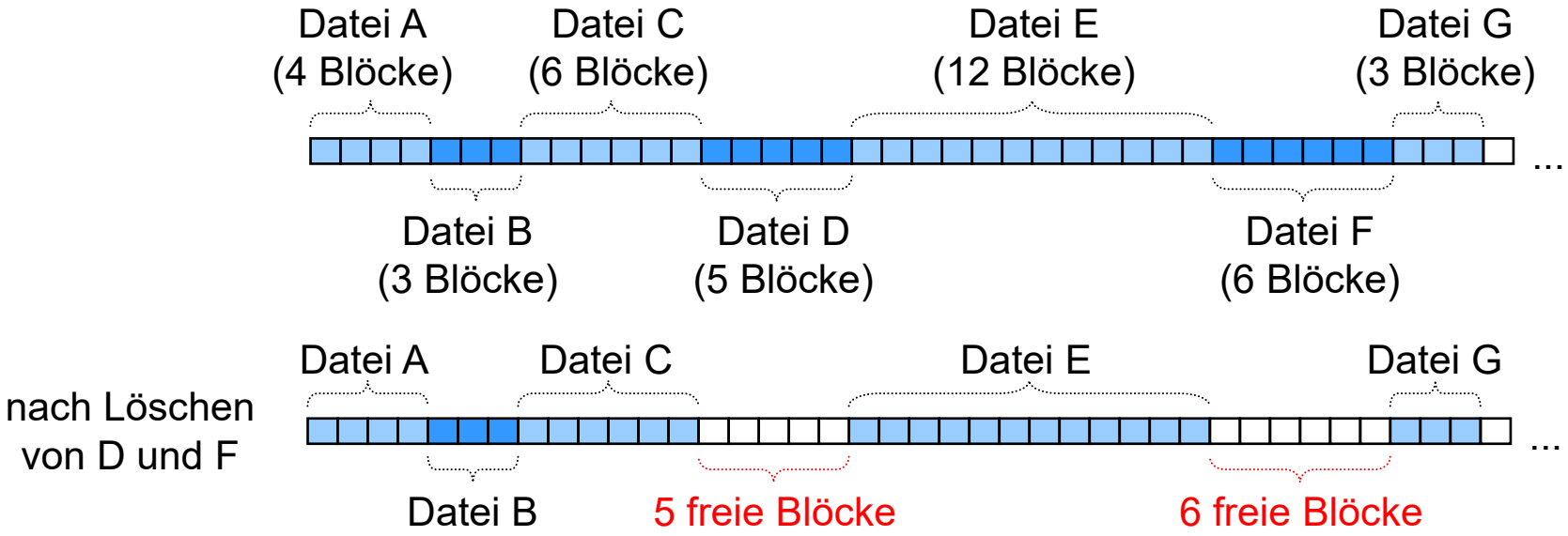
→ einfach zu implementieren

→ sehr gute Zugriffsleistung, da Datei in einem Stück ausgelesen werden kann

aber:

→ für Allokation von freien Blöcken muss maximale Dateigröße im Voraus bekannt sein

→ externe Fragmentierung des Datenträgers durch Löschen von Dateien



Allokationsverfahren (6)

Allokation mittels verketteten Listen

Speicherung einer Datei als verkettete Liste von Blöcken

→ erstes Wort in jedem Block ist ein Zeiger auf den nächsten Block

→ Rest des Blockes enthält die Daten

→ keine externe Fragmentierung

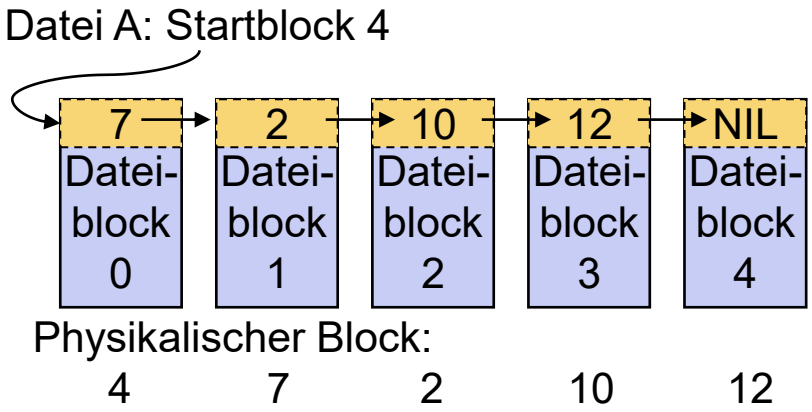
→ Verzeichnisse benötigen Adresse des 1. Blocks

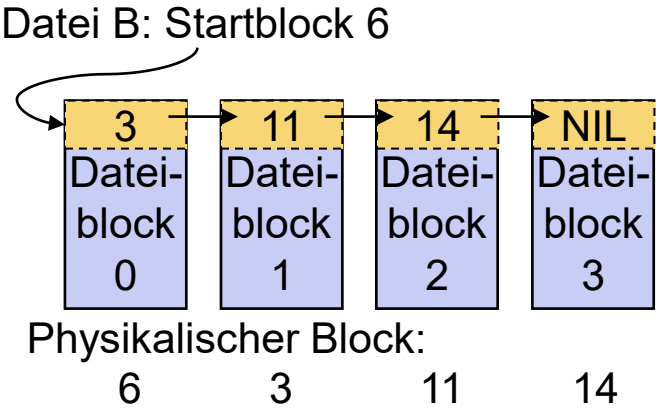
→ sequentieller Zugriff effizient möglich

aber:

→ wahlfreier Zugriff zeitaufwendig, da Blockliste nach Zielblock durchsucht werden muss

→ in einem Block wird keine Zweierpotenz an Bytes mit Nutzdaten abgelegt





Allokationsverfahren (7)

Datei-Allokationstabelle (File Allocation Table, FAT)

Tabelle mit Zeigern auf die einzelnen Blöcke einer Datei

→ Zeiger der verketteten Liste werden in FAT abgelegt

→ nutzbare Blockgröße ist wieder Zweierpotenz

→ FAT wird vor Dateizugriff in Hauptspeicher geladen

→ wahlfreier Zugriff effizient, da nur gesuchter Block von Platte zu lesen ist

→ Verzeichniseinträge benötigen weiterhin nur Plattenadresse des ersten Blocks

aber:

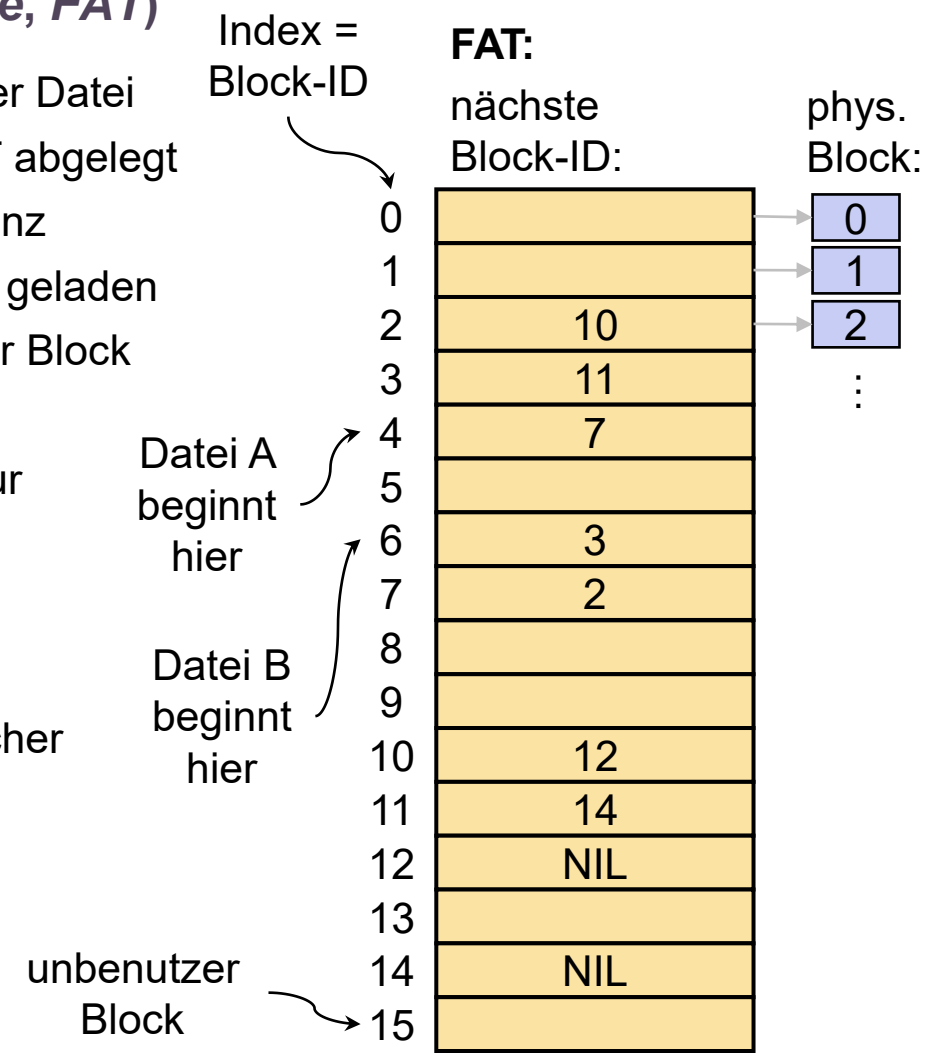
→ zusätzlicher Speicherbedarf im Hauptspeicher

Beispiel:

8 TB Festplatte, 16 KB Blockgröße

→ 2^{29} Einträge in FAT mit je 5 Byte

→ ca 2,5 GB Speicherbedarf für FAT



Allokationsverfahren (8)

Allokation mit *Index-Knoten* (*I-Nodes*)

eigene Datenstruktur **fester Größe** pro Datei,
welche die Plattenadressen und Attribute
der Dateiblöcke auflistet

→ I-Node muss nur dann im Hauptspeicher sein,
wenn die Datei geöffnet ist

→ geringerer Speicherbedarf im Hauptspeicher

aber:

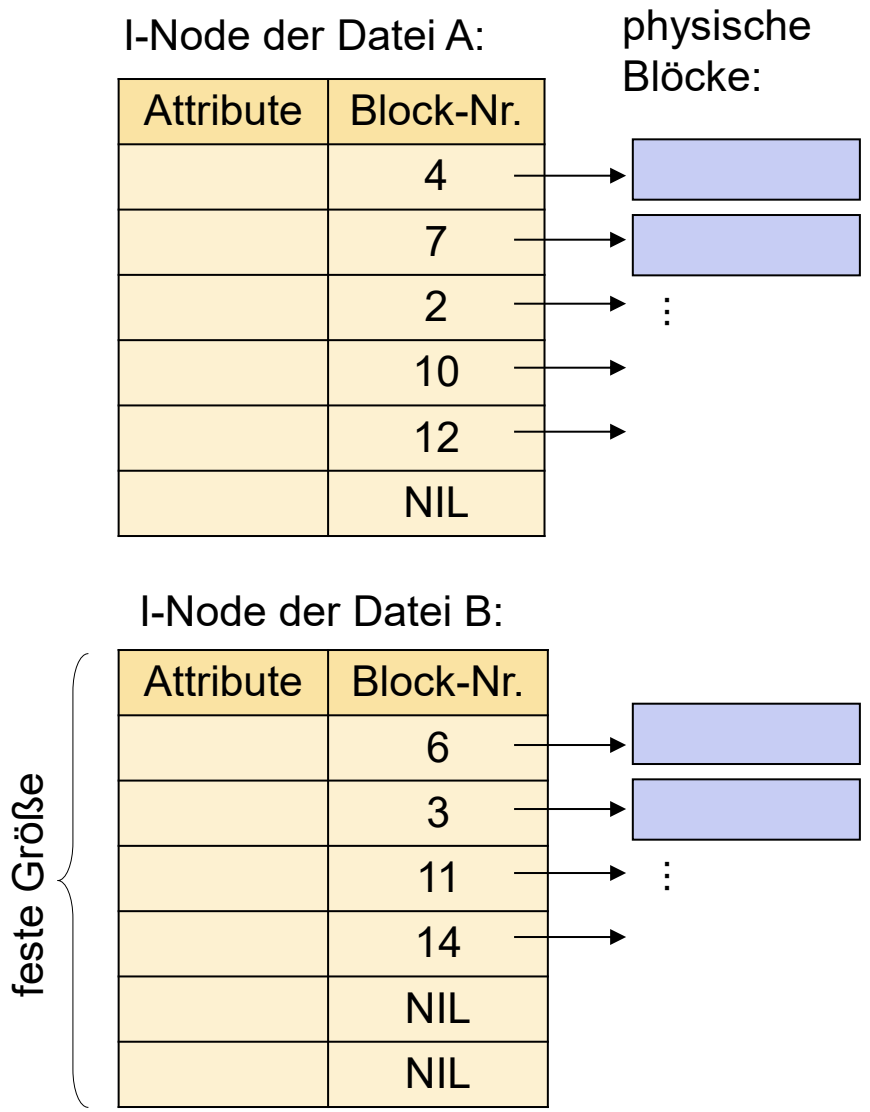
→ I-Nodes können nur begrenzt viele
Blockadressen aufnehmen

Praxis:

zumeist gibt es viele Dateien mit wenigen Blöcken
und wenige Dateien mit vielen Blöcken

→ I-Node für übliche Größen auslegen

→ Sonderlösung für große Dateien

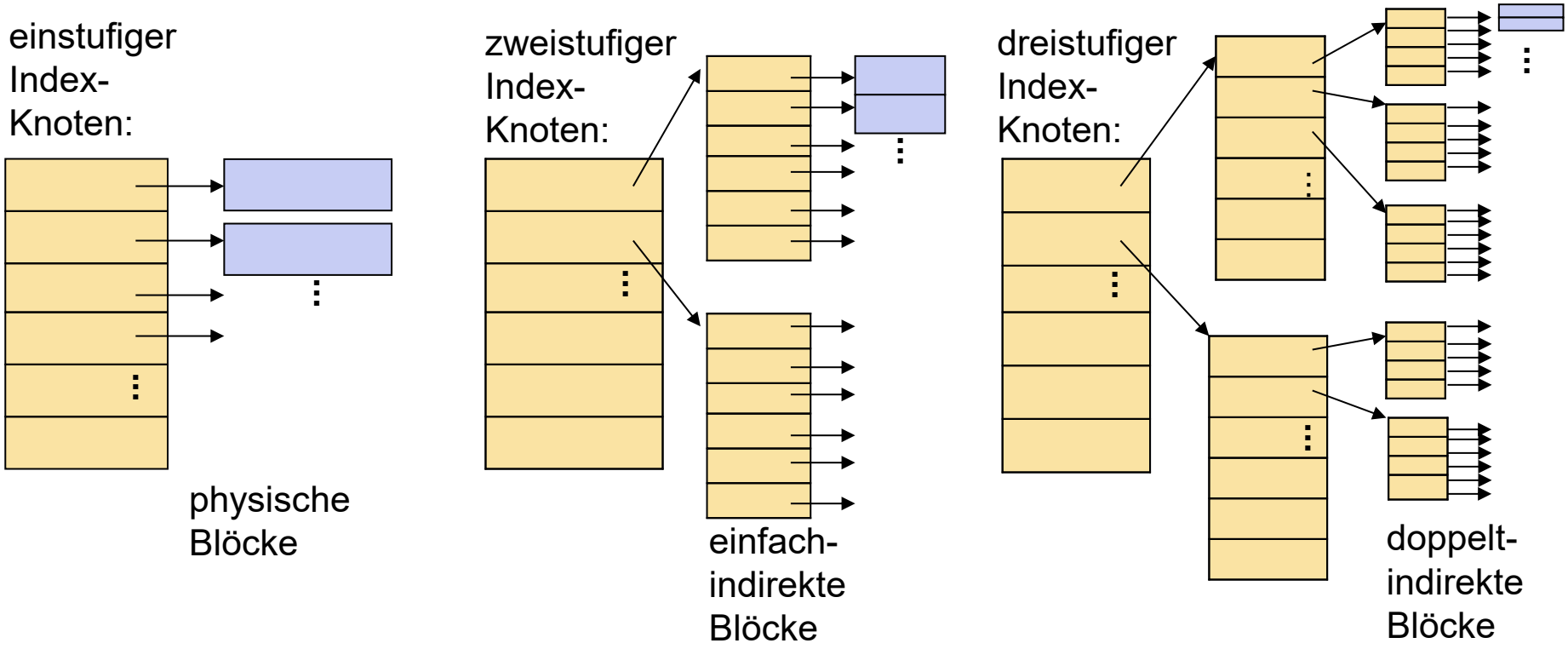


Allokationsverfahren (9)

Allokation mit mehrstufigen Index-Knoten

hierarchische Organisation der einfachen Index-Knoten in einem Baum

→ bei großen Dateien verweisen Index-Knoten-Einträge auf Blöcke, welche selbst Blockadressen enthalten



Vortragsübersicht

4 – Dateiverwaltung

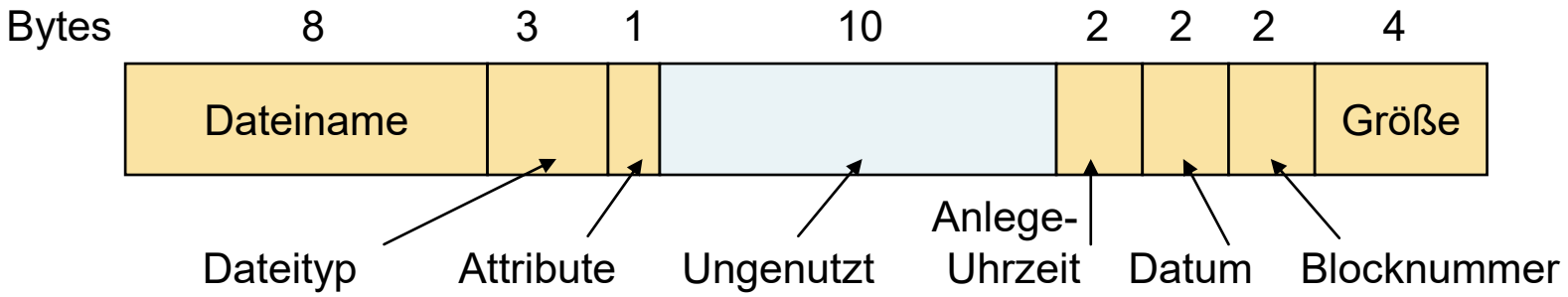
- Dateien
- Verzeichnisse
- Allokationsverfahren
- Praxisbeispiele

Praxisbeispiele (1)

Das Dateisystem von MSDOS

- kein Benutzerkonzept, d. h. kaum Schutz
- mehrstufige Verzeichnisstruktur ohne Querverweise (Links), d. h. Struktur ist ein Baum
- Verzeichniseinträge fester Größe (32 Byte)
- Dateinamen aus 8 + 3 Großbuchstaben
- Zeit der Datei-Erzeugung getrennt für Sek., Min., ..., Jahr
- exakte Dateigröße (max. 4 GB)
- erste Blocknummer verweist auf einen Eintrag einer FAT (max. 64K Einträge)
- unterstützte Dateiattribute z. B. "versteckt", "Systemdatei", "Verzeichnis", "Read-Only"

Verzeichniseintrag:



Praxisbeispiele (2)

Die FAT-Problematik

- FAT12, FAT16 und FAT32
  mit 12, 16 und 28 Bits (sic!) für Plattenadressen
- Plattenblockgröße ist Vielfaches von 512 Byte
- maximal 4 Partitionen unter MSDOS verwaltbar
- große Blockgrößen:
  → ermöglichen größere Partitionen
  → mehr Speicherverbrauch für kleine Dateien

Block-größe	FAT12	FAT16	FAT32
0,5 KB	2 MB		
1 KB	4 MB		
2 KB	8 MB	128 MB	
4 KB	16 MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

für Disketten

für Festplatten

für größere Festplatten

für Videos

begrenzte Sektorenadressen

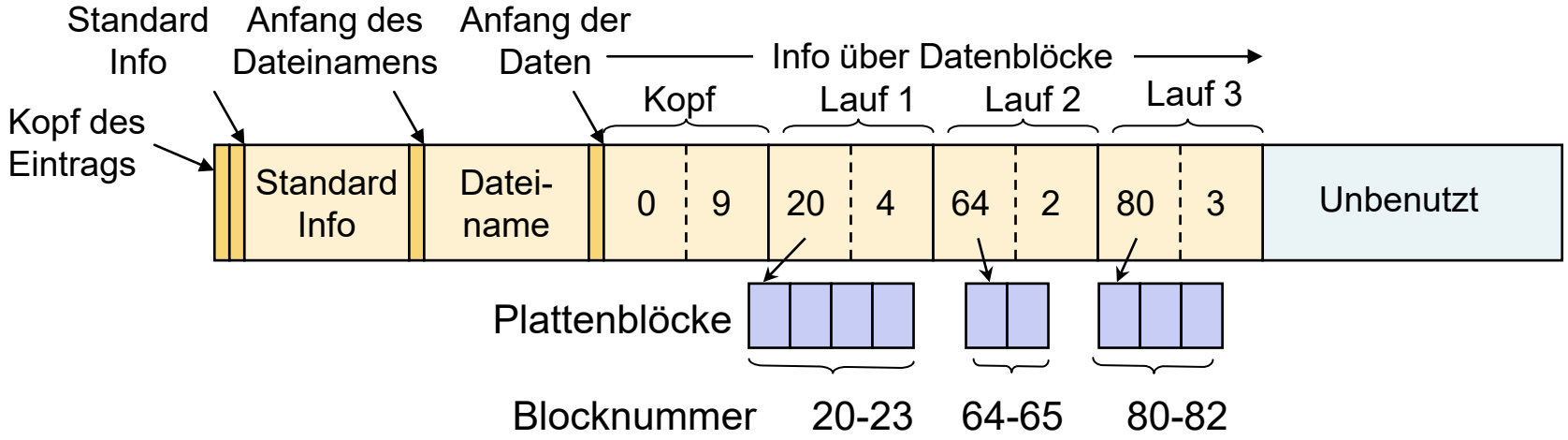
- exFAT: Erweiterung von FAT für Flash-Speicher
  → max. 16 ExaByte große Dateien
  → Rechtevergabe

Praxisbeispiele (3)

New Technology File System (NTFS)

- ab Windows NT bis heute
- Adressen mit 64 Bit und somit Partitionen bis zu 2^{64} Byte
- Dateien können aus mehreren Datenströmen (z. B. `file1:stream2`) bestehen
- Länge Gesamtpfad (mit Datei und Endung): ≤ 255 Unicode-Zeichen
  → ab Windows 10 beliebig lange Pfade möglich, mit Abstürzen bei Altsoftware
- Zugriffsrechte global und pro Datei (ACLs → VL-Einheit 5)
- *Master File Table* (MFT) mit Bitmap für freie Blöcke und MFT-Einträgen für Dateien

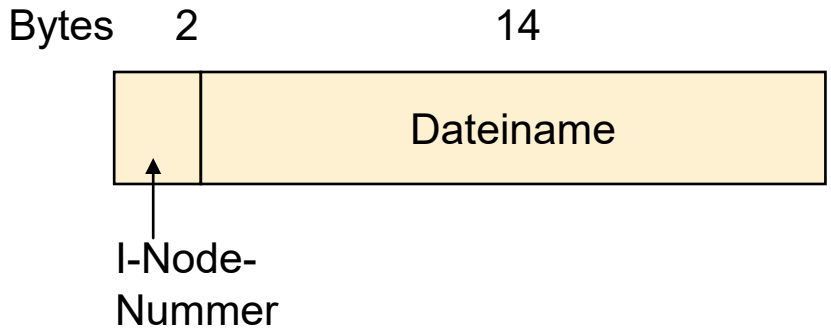
MFT-Eintrag:



Praxisbeispiele (4)

Verzeichnisse in UNIX

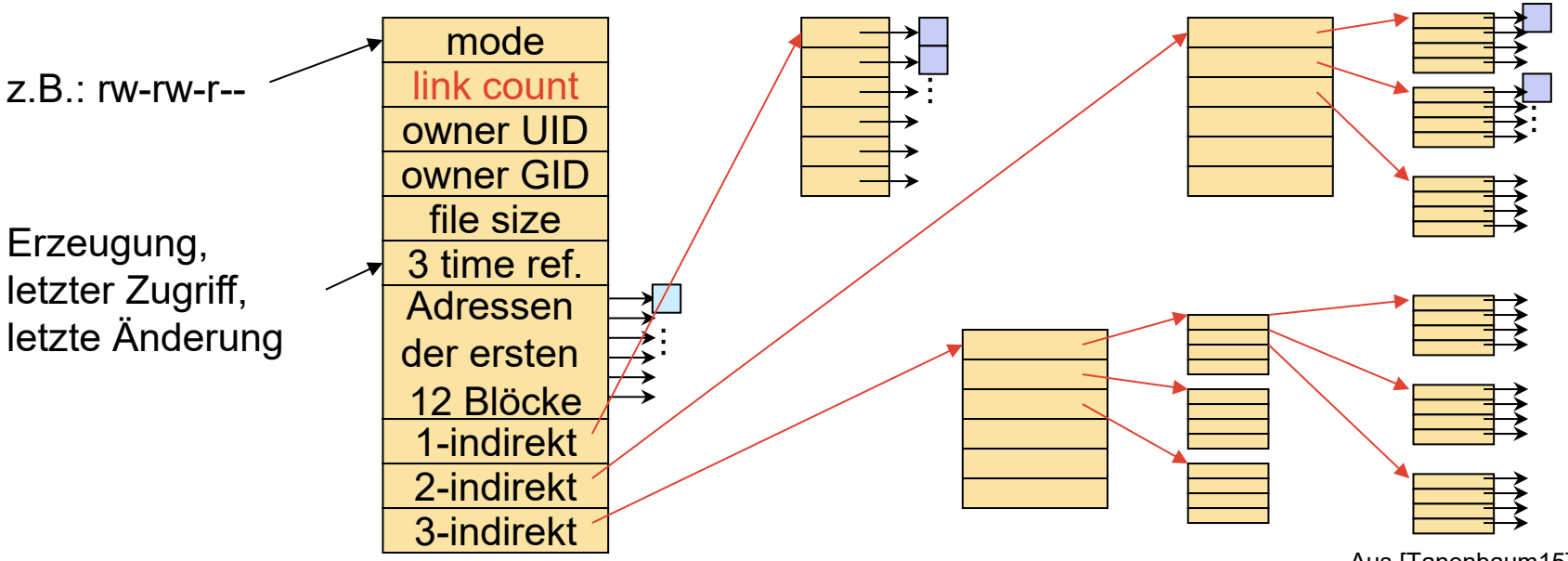
- extrem einfache Verzeichnisstruktur:
  → jeder Eintrag enthält nur den Dateinamen und die I-Node-Nummer
  → alle weiteren Informationen (z. B. Typ, Größe, Zeiten, Plattenblöcke) liegen im I-Node
- beim Zugriff auf eine Datei lokalisiert das OS die Plattenblöcke zum gegebenen Dateinamen
- I-Node des Wurzelverzeichnisses ist auf einer fixierten Stelle auf der Partition positioniert



Praxisbeispiele (5)

Dateien in ext2

- im Index-Knoten werden relevante Information für kurze und für lange Dateien gehalten
- Inhalt von Index-Knoten ist in konkreten OS erweitert
  (z. B. in System V mit Semaphor-Information zur Sperrung der Datei)
- Hauptindexknoten enthält allgemeine Infos (z. B. Name)
  und Adressen der ersten 12 Datenblöcke
  → Verweis auf weitere Blöcke mit einfach-, zweifach-, dreifach-indirekten Blöcken

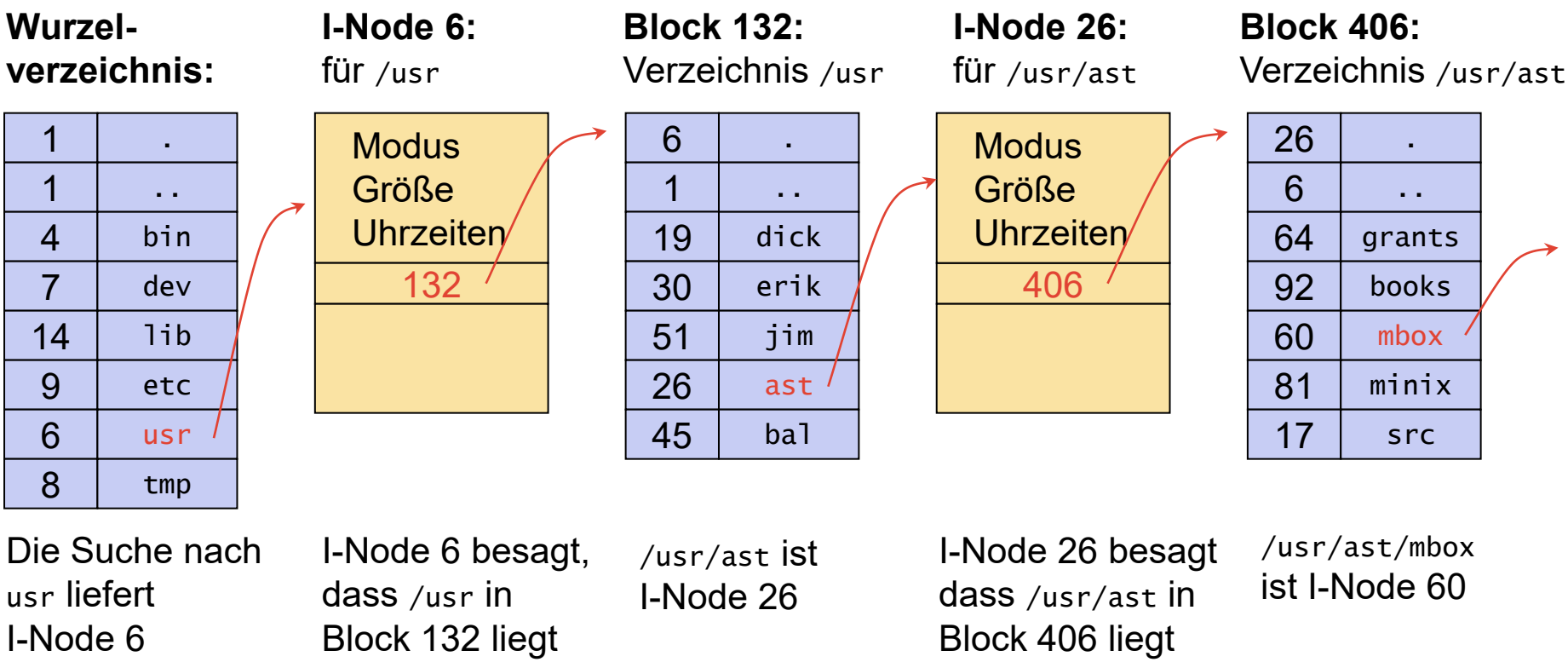


Aus [Tanenbaum15]

Praxisbeispiele (6)

Dateizugriffe in UNIX

- angenommen, man wolle auf `/usr/ast/mbox` zugreifen
- relative Pfadnamen analog, aber Start im aktuellen Arbeitsverzeichnis



Praxisbeispiele (7)

Journaling-Dateisystem

Dateisystem, welches Aktionen in einem Journal aufzeichnet,

ehe sie ausgeführt werden

→ nicht ausgeführte Aktionen (z. B. durch Absturz) werden nachgeholt

→ robust gegenüber Abstürzen

→ Implementierung z. B. in NTFS, ext4, ReiserFS, OS X

Rahmenbedingungen:

- jede Aktion im Journal muss *idempotent* (bei wiederholter Ausführung effektfrei) sein
  (z. B.: "markiere Block 127 als frei")
- zusätzliche Robustheit durch atomare Aktionen erreichbar
  (z. B.: "Schreibaktion wird entweder ganz oder gar nicht durchgeführt")

Allokationsverfahren (8)

Verbreitete Dateisysteme im Überblick

	Ent-wickler	Jahr	OS	Datei-namen	Datei-pfad	Max. Datei-größe*	Max. Speicher-kapazität*	Datei-Links	Journal
Fat16	Microsoft	1987	DOS 3.31	8.3	?	2 GB	16 MB…2 GB	–	no
ext2	R. Card	1993	Linux	< 255	∞	16 GB…2 TB	2…32 TB	H, S	no
NTFS	Microsoft	1993	Win NT	< 255	< 32 M	16 EB	16 EB	S	yes
Fat32	Microsoft	1996	Win 95b	8.3	∞	4 GB	0,5…8 TB	–	no
HFS+	Apple	1998	Mac OS 8.1	< 255	∞	8 EB	8 EB	S	yes
ISO 9660	Microsoft	1999	Viele	8.3…180	< 180	4 GB…8 TB	8 TB	–	?
ext4	versch.	2006	Linux	< 255	∞	16 GB…16 TB	1 EB	H, S	yes
GFS 2	Red Hat	2006	Linux	< 255	∞	2 TB…8 EB	2 TB…8 EB	H, S	yes
exFAT	Microsoft	2006	Win CE 6.0	< 255	∞	16 EB	64 ZB	H, S	no
APFS	Apple	2016	MacOS, iOS, Watch OS	?	?	8 EB	?	?	?

[https://en.wikipedia.org/wiki/Comparison_of_file_systems]

Zusammenfassung

Dateien und Verzeichnisse

- Begriffe: Datei, Dateisystem, Dateiname, Dateiendung, ...
- Dateiarten, Dateiattribute
- Ordnung durch (i. d. R. hierarchische) Verzeichnisse
- Referenzierung über absolute oder relative Pfadangaben

Umsetzung und Praxis

- Datenträger mit Partitionen, ein Dateisystem pro Partition
- grundlegende Allokationsverfahren: kontinuierlich, verkettete Listen
- FATs speichern Verweise aus verketteter Liste in separater Tabelle